

COURSE 2A

C2-05 PYTHON AUTOMATION FOR ANALYSTS

BEGINNER-FIRST TEACHING BOOK - RC1

Scripts -> pandas -> files/APIs -> defensive runs -> logging -> reproducibility

THIS BOOK IS YOUR TEACHER

Technical target: Python 3.14.7 + pandas 3.0.5 references verified 2026-09-10.

This book is your teacher

Use one lesson at a time. New advanced concepts are still taught from first principles. Save a genuine attempt before opening protected review; do not skim ahead just because a topic name looks familiar.

Daily lesson loop

1) Why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it in your head -> 5) new words -> 6) watch / visual source when useful -> 7) follow with me once -> 8) expected result -> 9) why it worked -> 10) common beginner mistakes -> 11) recovery ladder -> 12) small fresh task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence / mastery + exact next route.

Source rule

External videos are companions, not the course authority. If a useful source exists, open the exact link and return to the course task. If an external source would leak a solved capstone or add little value, the lesson explicitly uses an INTERNAL TEACHING ROUTE.

Attempt protection

Do the teacher example once. Then close it and complete the changed task from your own blank workbook/query/report/notebook. Protected review opens only after a saved attempt and is followed by a changed retry.

Mastery standard

You are finished only when you can reproduce the result, validate it, explain why it works, recover from a mistake and save the required evidence.

Contents / Index

Every canonical C2-05 lesson and actual page number

This book is your teacher	2
Contents / Index	3
Prerequisite and tool map	4
Practice, review and Gate route	5
PYA1 Writing reusable analyst scripts	6
PYA2 Functions and modular code	10
PYA3 Robust pandas transformations	14
PYA4 Multi-file processing	18
PYA5 Excel automation workflows	22
PYA6 CSV and JSON handling	26
PYA7 APIs for analysts	30
PYA8 Error handling and defensive scripts	34
PYA9 Logging and validation checks	38
PYA10 Automated recurring reporting	42
PYA11 Reproducible analytical pipelines	46
Module mastery checklist + quick reference	50
Source / video registry	51

Prerequisite and tool map

What Course 1 knowledge is assumed and what C2-05 adds

ASSUMED FROM COURSE 1

Basic Python syntax, functions, lists/dictionaries, pandas DataFrames, filtering/groupby/merge basics and reading CSV. C2-05 does not reteach Python from zero, but every NEW automation/reliability concept is explained from first principles.

TOOLS

Python 3.14.x target; pandas 3.0.x target; Requests for API wrapper; pathlib/json/csv/logging/venv from standard library. Excel practice may use a pandas-compatible engine. Jupyter/VS Code are optional authoring surfaces.

NEW DEPTH

Clean-start scripts, modular contracts, pandas reliability, multi-file lineage, round-trip Excel checks, nested JSON grain, defensive APIs, safe exceptions, logging/validation, rerun-safe reporting and environment/run manifests.

OPTIONAL-PATH POSITION

C2-05 is optional Senior/BI Analyst depth. Its reliability habits overlap strongly with C2B/C3, but a direct DA-to-DE learner follows Course 1 -> C2B -> C3. C2A cannot silently become a prerequisite.

Practice, review and Gate route

Clean run -> failure injection -> changed transfer -> evidence



LESSON RULE

Each lesson has a matching P-PYA practice section. Save runnable evidence and at least one validation/failure result before opening its protected review. Close the review before the changed retry.

MINI-LAB

After PYA1-PYA11, build a Weekly Analyst Reporting Pipeline from raw monthly files to validated output, log, validation JSON and manifest. Run it twice and inject one failure.

GATES

Gate A = Retail Orders Automation. Gate B = Support Ticket Automation. Gate C = Subscription Billing Automation. Different fixtures/seeds/failure traps; B/C stay protected full reassessments.

PYA1 - Writing reusable analyst scripts

1. Why it matters

A notebook can be perfect for exploration and still be fragile as a recurring business process. Automation starts when the workflow has explicit inputs, an entry point and predictable outputs.

2. What you are learning

Move a one-off notebook flow into a small script with pathlib paths, functions, explicit arguments and a main entry point. Prove that the workflow works from a clean process.

3. Prerequisite / what you already know

Course 1 Python/pandas basics: variables, functions, DataFrames, read_csv and simple transformations.

4. Picture it in your head

Mental model

Imagine this as a visible path: Notebook idea -> Explicit inputs -> Functions -> main() -> Outputs -> Clean rerun. Keep the stages separate. If the result becomes wrong, walk backward one stage at a time until you find where the meaning, rule, grain, relationship, or control changed.

5. New words before we use them

Entry point: The place where a script intentionally starts its workflow. Hidden state: Data or variables left behind by earlier notebook cells that later code silently depends on. Deterministic run: A run that produces the same intended business result from the same approved inputs/configuration.

6. Watch / visual source

VISUAL/SOURCE ROUTE: Real Python short visual: file-path problem/pathlib overview; Corey Schafer 2024 pathlib is optional reinforcement.

<https://docs.python.org/3.14/tutorial/modules.html>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Use Path objects for input/output locations rather than hand-built string paths.
- 2 Create `load_sales(path)`, `clean_sales(df)`, `validate_sales(df)` and `write_output(df,path)`. Each function gets what it needs through arguments.
- 3 Create `main(input_path, output_path)` that orchestrates the smaller functions.
- 4 Guard the command-line entry with `if __name__ == '__main__':` so importing the module does not unexpectedly run the business workflow.
- 5 Restart the Python process and run again. Then remove/rename the required input once to prove the error identifies the missing file before final output is written.

8. What you should see / be able to say

Expected result

A clean-start script that can be rerun without notebook cell-order dependence.

9. Why it worked

Reasoning

Inputs and dependencies are visible in function signatures and `main()`, so the run no longer relies on invisible notebook memory. Professional upgrade: Use configuration/arguments for paths and business period. Keep orchestration thin and transformation functions testable. A second analyst should know where execution starts.

10. Common beginner mistakes

1. Copying the entire notebook into one long script. | 2. Hard-coding personal desktop paths inside cleaning logic. | 3. Using global DataFrames that functions silently mutate. | 4. Writing final output before required-input validation passes.

11. If you get stuck - recovery ladder

Recovery

12. Now you do a small version alone

Fresh independent task

Fresh case: refactor a customer-churn notebook into `load_data()`, `normalize_status()`, `validate_data()`, `summarize()`, `write_output()` and `main()`.

13. Validate your result

Validation

Run from a clean process twice; input file unchanged; output row/total controls stable; missing input produces an actionable failure.

14. Teach it back in your own words

Explain without reading

Explain why a script that works only after notebook cell 3 was run is not repeatable automation.

15. Protected review + changed fresh retry

Attempt protection

Save the first script and clean-run evidence before review. Changed retry uses a different input folder/output name through configuration only.

16. Evidence / mastery + exact next route

Mastery evidence

Script/notebook + clean-run proof + missing-input failure + rerun control + explain-back. After saving evidence, continue to PYA2.

PYA2 - Functions and modular code

1. Why it matters

One giant automation function is hard to test and risky to change. Small functions let you repair one behavior without breaking unrelated file discovery, transformation or delivery logic.

2. What you are learning

Design functions with one coherent responsibility, explicit parameters, useful return values, docstrings and lightweight deterministic assertions.

3. Prerequisite / what you already know

PYA1 script entry-point structure and basic function syntax.

4. Picture it in your head

Mental model

Imagine this as a visible path: One job -> Inputs -> Return value -> Failure rule -> Small test. Keep the stages separate. If the result becomes wrong, walk backward one stage at a time until you find where the meaning, rule, grain, relationship, or control changed.

5. New words before we use them

Function contract: Accepted inputs, returned outputs, assumptions and failure behavior. Pure transformation: A function that transforms supplied data without hidden external side effects. Assertion: A check that stops or flags execution when an expected condition is false.

6. Watch / visual source

VISUAL/SOURCE ROUTE: Book-led; Python modules reference is sufficient.

<https://docs.python.org/3.14/tutorial/modules.html>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Create `normalize_channel(series)` that returns a new normalized Series instead of changing a global DataFrame.
- 2 Create `require_columns(df, required)` that raises `ValueError` listing missing columns.
- 3 Create `summarize_by_region(df)` whose output grain is one row per region.
- 4 Add one normal assertion and one edge-case assertion to each critical helper where practical.
- 5 Keep file reading/writing outside pure transformations so tests do not need the whole filesystem.

8. What you should see / be able to say

Expected result

Small composable functions with visible contracts and simple repeatable tests.

9. Why it worked

Reasoning

Each function can be tested in isolation with tiny deterministic inputs. Professional upgrade: Module boundaries matter more as analyst automation grows. Keep transformations reusable and orchestration responsible for I/O and sequencing.

10. Common beginner mistakes

1. Function does discovery, cleaning, joins, charts and writes in one block. | 2. Returns nothing and mutates hidden global state. | 3. Catches its own error and returns an empty DataFrame. | 4. Docstring says what code does but omits assumptions/failure behavior.

11. If you get stuck - recovery ladder

Recovery

Circle each verb in the giant function. Each distinct responsibility is a candidate function boundary. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: create `normalize_priority()`, `validate_required_columns()`, `calculate_net_value()` and `summarize_team()`.

13. Validate your result

Validation

At least two assertions include a normal and edge/failure case; function outputs have stated grain.

14. Teach it back in your own words

Explain without reading

Explain a function contract using input -> promise -> output/failure.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry adds a new channel alias without modifying file-loading/reporting functions.

16. Evidence / mastery + exact next route

Mastery evidence

Four functions + docstrings/contracts + assertions + explain-back. After saving evidence, continue to PYA3.

PYA3 - Robust pandas transformations

1. Why it matters

A pandas transformation can silently produce plausible values when dates/numbers were read as text or malformed records were coerced away. Reliable automation treats dtypes and business calculations as contracts.

2. What you are learning

Parse dates/numerics deliberately, use explicit assignment under modern pandas behavior, calculate metrics vectorially and reconcile results to independent controls.

3. Prerequisite / what you already know

Course 1 pandas filtering/grouping and PYA2 functions.

4. Picture it in your head

Mental model

Imagine this as a visible path: Schema -> Parse types -> Transform -> Reconcile -> Exception rows. Keep the stages separate. If the result becomes wrong, walk backward one stage at a time until you find where the meaning, rule, grain, relationship, or control changed.

5. New words before we use them

dtype contract: The expected data type for each important column. Vectorized operation: A pandas/NumPy-style column operation applied across rows without manual Python loops. Coercion: Converting invalid values into another representation such as NaN/NaT rather than failing.

6. Watch / visual source

VISUAL/SOURCE ROUTE: Book-led with current pandas docs; IO video later reinforces file handling.

https://pandas.pydata.org/docs/user_guide/

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Load sales CSV and record raw row count.
- 2 Parse order_date with errors='raise'; parse quantity/unit_price/discount_pct as numeric with deliberate error policy.
- 3 Normalize channel using explicit assignment; avoid chained-assignment patterns.
- 4 Calculate gross_sales and calc_net_sales using vectorized columns.
- 5 Create validation_diff against supplied net_sales and fail/quarantine rows beyond the approved tolerance.

8. What you should see / be able to say

Expected result

Typed DataFrame with explicit exception behavior and reconciled calculated metrics.

9. Why it worked

Reasoning

Bad types cannot silently flow through the same path as valid business records. Professional upgrade: pandas 3.x Copy-on-Write and evolving APIs make explicit assignments and current documentation preferable to old notebook habits.

10. Common beginner mistakes

1. Leaving numeric fields as object/string. | 2. Using errors='coerce' without counting/quarantining new missing values. | 3. Chained assignment on filtered slices. | 4. Checking only aggregate total while individual bad rows cancel each other.

11. If you get stuck - recovery ladder

Recovery

Inspect dtypes and malformed examples first. Parse one critical column, count failures, then continue. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: transform service transactions with mixed date/numeric formats and one malformed price.

13. Validate your result

Validation

No unexpected object dtype; bad-record policy visible; per-row reconciliation within tolerance; aggregate total reconciles.

14. Teach it back in your own words

Explain without reading

Explain why 'pandas read the file successfully' is not a data-quality PASS.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry uses changed row order and one new malformed value.

16. Evidence / mastery + exact next route

Mastery evidence

Transformation function + dtype report + exception list + row/total reconciliation + explain-back. After saving evidence, continue to PYA4.

PYA4 - Multi-file processing

1. Why it matters

Manual copy/paste across monthly files is slow and unsafe. Automated discovery is only trustworthy when file scope, schema compatibility and row lineage are controlled.

2. What you are learning

Discover files with pathlib, sort deterministically, exclude backup/drift files deliberately, preserve source_file lineage and reconcile per-file contributions.

3. Prerequisite / what you already know

PYA1 pathlib/script structure and PYA3 pandas transformations.

4. Picture it in your head

Mental model

Imagine this as a visible path: Expected files -> Discover -> Schema check -> Lineage -> Concat -> Reconcile. Keep the stages separate. If the result becomes wrong, walk backward one stage at a time until you find where the meaning, rule, grain, relationship, or control changed.

5. New words before we use them

File-discovery contract: Rules defining which files are expected, included and excluded. Lineage: Evidence showing which source file produced each row/output contribution. Schema drift: Unexpected change in column names/types/shape between files.

6. Watch / visual source

VISUAL/SOURCE ROUTE: Recommended visual: pathlib file discovery; use current docs for API behavior.

<https://docs.python.org/3.14/library/pathlib.html>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Use sorted(Path(input_dir).glob('sales_2026_*.csv')) but compare matches with an expected-file rule before loading.
- 2 Exclude files only through explicit contract, not because their values look strange.
- 3 Load each approved file, verify required columns, add source_file=file.name and record its row count.
- 4 Concatenate approved parts with ignore_index=True.
- 5 Prove combined rows equal sum of per-file rows and produce a file contribution table.

8. What you should see / be able to say

Expected result

One traceable combined dataset with deterministic file scope and schema checks.

9. Why it worked

Reasoning

Every combined row retains a source identity and file-level controls catch missing/extra contributions. Professional upgrade: File discovery, lineage and schema checks are small-scale ingestion engineering patterns and transfer directly to DE work.

10. Common beginner mistakes

1. Glob pattern accidentally ingests backup files. | 2. Hard-coded list requires code edit every month. | 3. Concatenating incompatible schemas and accepting NaN-filled new columns silently. | 4. Dropping source filename before validation.

11. If you get stuck - recovery ladder

Recovery

Print expected vs actual filenames first. Do not read data until the file-scope mismatch is classified. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: process four support-ticket exports where one backup and one schema-drift file are present.

13. Validate your result

Validation

Expected/actual file list, schema report, per-file rows and combined row parity all saved.

14. Teach it back in your own words

Explain without reading

Explain why 'glob found 5 files' is not enough to know whether 5 files should be processed.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry switches to a new directory with a different approved month count.

16. Evidence / mastery + exact next route

Mastery evidence

Discovery function + file-scope table + schema checks + lineage + reconciliation + explain-back. After saving evidence, continue to PYA5.

PYA5 - Excel automation workflows

1. Why it matters

Excel is a common analyst delivery format, but an attractive workbook can still contain wrong totals. Automation must treat the spreadsheet as an output contract and verify it after writing.

2. What you are learning

Create a fresh Excel output from pandas results, preserve raw/template files, use deterministic sheet/column names and re-read exported values for reconciliation.

3. Prerequisite / what you already know

PYA3 transformations and basic Excel familiarity.

4. Picture it in your head

Mental model

Imagine this as a visible path: DataFrame -> Validated summary -> Write new file -> Re-open -> Reconcile -> Deliver. Keep the stages separate. If the result becomes wrong, walk backward one stage at a time until you find where the meaning, rule, grain, relationship, or control changed.

5. New words before we use them

Output contract: The required workbook/sheet/column structure and values expected by downstream users. Round-trip check: Writing an output, re-reading it and reconciling against the source calculation. Template preservation: Keeping the provided raw/template workbook unchanged and producing a new output file.

6. Watch / visual source

VISUAL/SOURCE ROUTE: Corey Schafer pandas IO: CSV/Excel sections are useful visual support.

https://pandas.pydata.org/docs/user_guide/io.html

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Build a validated regional summary DataFrame from the source truth.
- 2 Write a NEW workbook with a stable sheet name such as Regional Summary. Do not overwrite practice/template input.
- 3 Close the writer/process so the file is complete on disk.
- 4 Read the output sheet back with pandas.
- 5 Compare row count, region set and net-sales total to the source summary before declaring delivery PASS.

8. What you should see / be able to say

Expected result

An Excel report whose saved values reconcile to the DataFrame that produced it.

9. Why it worked

Reasoning

The last-mile file is tested independently rather than trusted because writing did not raise an exception. Professional upgrade: For recurring delivery, calculation truth should live in validated code/model logic; Excel is a controlled presentation/output surface.

10. Common beginner mistakes

1. Overwriting raw input/template. | 2. Hard-coded row numbers and manual formulas that break when regions change. | 3. No re-read after export. | 4. Visual formatting used to distract from a failed total.

11. If you get stuck - recovery ladder

Recovery

Strip to one output sheet and values only. Prove round-trip parity before adding presentation formatting. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: create Team Summary and Exceptions sheets from support data and verify both after writing.

13. Validate your result

Validation

Template unchanged; expected sheets/columns; row and total parity; output opens after process ends.

14. Teach it back in your own words

Explain without reading

Explain why successful `to_excel()` does not prove the workbook is correct.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry adds a new region/team without editing manual row formulas.

16. Evidence / mastery + exact next route

Mastery evidence

New xlsx output + round-trip checks + template hash/unchanged note + explain-back. After saving evidence, continue to PYA6.

PYA6 - CSV and JSON handling

1. Why it matters

CSV is flat, JSON may be nested, and careless normalization can multiply order-level values across item rows. File format handling is really a grain problem plus encoding/schema discipline.

2. What you are learning

Read/write CSV/JSON safely, inspect nested structure, declare output row grain before normalization and reconcile nested item counts after flattening.

3. Prerequisite / what you already know

PYA3 pandas transforms and grain concepts from C2-00/C2-03.

4. Picture it in your head

Mental model

Imagine this as a visible path: Format -> Nested shape -> Declare grain -> Normalize -> Export -> Reconcile. Keep the stages separate. If the result becomes wrong, walk backward one stage at a time until you find where the meaning, rule, grain, relationship, or control changed.

5. New words before we use them

Nested JSON: JSON containing objects/lists inside other objects rather than one flat row structure. Normalization: Transforming nested records into a tabular structure. Encoding: The character representation used when reading/writing text files such as UTF-8.

6. Watch / visual source

VISUAL/SOURCE ROUTE: Corey Schafer pandas IO: JSON sections; current Python json/pandas docs control syntax.

<https://docs.python.org/3.14/library/json.html>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Load nested_orders.json with UTF-8 and inspect one order before flattening.
- 2 Declare output grain: one row per order item.
- 3 Use json_normalize/controlled list expansion to carry order_id and customer fields onto item rows.
- 4 Do not copy order_total into an item-level revenue measure unless its repeated meaning is intentional.
- 5 Count total nested items independently and reconcile to flattened row count; export deterministic columns.

8. What you should see / be able to say

Expected result

Flat CSV/DataFrame whose row meaning and count exactly match the declared nested grain.

9. Why it worked

Reasoning

The grain is decided before flattening, preventing accidental double counting. Professional upgrade: Semi-structured data is common in APIs and event systems; explicit grain and lineage are foundational pipeline habits.

10. Common beginner mistakes

1. Assuming JSON is already a table. | 2. Flattening one level too far/too shallow. | 3. Repeating order total on every item then summing it. | 4. Ignoring optional/missing nested fields.

11. If you get stuck - recovery ladder

Recovery

Draw one JSON object. Circle the repeating list. Decide whether one output row represents parent, child or combination before coding. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: flatten support cases containing a list of status events; preserve case/customer fields without overstating case-level SLA target.

13. Validate your result

Validation

Flattened row count equals nested event count; parent IDs retained; missing optional field handled explicitly.

14. Teach it back in your own words

Explain without reading

Explain why JSON normalization is a row-grain decision, not just a pandas function call.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry removes one optional field from a record.

16. Evidence / mastery + exact next route

Mastery evidence

Normalization code + written grain + nested/flat count reconciliation + export + explain-back. After saving evidence, continue to PYA7.

PYA7 - APIs for analysts

1. Why it matters

Replacing a manual download with an API call can make a process faster but also introduces network failures, HTTP errors and changing payloads. A weekly job must not hang forever or accept an error payload as successful data.

2. What you are learning

Explain request/response flow; write a defensive Requests wrapper with timeout, `raise_for_status` and bounded exception translation; keep lesson mastery deterministic using local JSON.

3. Prerequisite / what you already know

PYA6 JSON handling and basic functions/exceptions.

4. Picture it in your head

Mental model

Imagine this as a visible path: Request -> Timeout -> Status -> JSON -> Parse -> Validate. Keep the stages separate. If the result becomes wrong, walk backward one stage at a time until you find where the meaning, rule, grain, relationship, or control changed.

5. New words before we use them

HTTP status: Numeric response status indicating request outcome such as success/client/server error. Timeout: Limit on how long a request waits for network progress before raising an error. `raise_for_status`: Requests method that raises an `HTTPError` for unsuccessful HTTP status responses.

6. Watch / visual source

VISUAL/SOURCE ROUTE: Corey Schafer Requests tutorial is visual support; current Requests docs govern reliability.

<https://requests.readthedocs.io/en/latest/user/quickstart/>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 First parse the supplied local `api_response.json` so network availability cannot block mastery.
- 2 Define `fetch_json(url, timeout=15)`. Always pass an explicit timeout.
- 3 Call `response.raise_for_status()` before trusting `response.json()`.
- 4 Translate `Timeout` and `RequestException` with useful URL/context while preserving the original exception chain.
- 5 Do not embed API keys/tokens in notebook/source evidence; use environment/config secret mechanisms in real work.

8. What you should see / be able to say

Expected result

A wrapper whose success and failure paths are explicit plus a network-independent parsing exercise.

9. Why it worked

Reasoning

Transport success, HTTP success and JSON parsing are treated as separate conditions. Professional upgrade: Requests documentation explicitly recommends timeouts for production code; reliability belongs in the wrapper, not every downstream parser.

10. Common beginner mistakes

1. `requests.get(url)` with no timeout. | 2. Calling `.json()` and assuming HTTP success. | 3. `except Exception: return {}`. | 4. Hard-coding credentials in source.

11. If you get stuck - recovery ladder

Recovery

Test parsing with local JSON first. Then test wrapper using a harmless endpoint only if network is available. Inject a fake/bad status or mock/local failure path. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: design a wrapper for a customer-status API and a parser that extracts one row per customer status record.

13. Validate your result

Validation

Local practice passes offline; timeout specified; HTTP failures cannot become valid data; credentials absent.

14. Teach it back in your own words

Explain without reading

Explain why valid JSON can still represent a failed API response.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry changes endpoint/base URL through config only.

16. Evidence / mastery + exact next route

Mastery evidence

Wrapper + local parser + error-path evidence + security note + explain-back. After saving evidence, continue to PYA8.

PYA8 - Error handling and defensive scripts

1. Why it matters

The most dangerous automation failure is not always a crash; it is a script that swallows an error and publishes a plausible but wrong report.

2. What you are learning

Catch specific exceptions you can contextualize/recover from, re-raise with context, distinguish data-quality failure from technical failure and block final output when trust controls fail.

3. Prerequisite / what you already know

PYA1-PYA7 workflow pieces.

4. Picture it in your head

Mental model

Imagine this as a visible path: Failure -> Classify -> Add context -> Stop/quarantine -> Evidence -> Rerun. Keep the stages separate. If the result becomes wrong, walk backward one stage at a time until you find where the meaning, rule, grain, relationship, or control changed.

5. New words before we use them

Exception chaining: Raising a clearer higher-level exception while preserving the original cause with 'from exc'. Quarantine: Separating bad records from valid processing with evidence rather than silently dropping them. Fail fast: Stop at the earliest point where continuing would make the result unsafe.

6. Watch / visual source

VISUAL/SOURCE ROUTE: Book-led; Python Errors and Exceptions is authoritative.

<https://docs.python.org/3.14/tutorial/errors.html>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Create `require_columns()` that raises `ValueError` with the exact missing list.
- 2 Wrap file loading only to add path/business context; preserve `FileNotFoundError` or chain a meaningful error.
- 3 Do not catch an exception unless you can recover or add useful context.
- 4 Treat failed key uniqueness/reconciliation as a business-quality blocker even if no Python exception happened.
- 5 Write final business output only after critical validation returns PASS.

8. What you should see / be able to say

Expected result

A workflow that either publishes trusted output or stops/quarantines with clear evidence.

9. Why it worked

Reasoning

Failure paths are designed rather than accidentally hidden by broad catches. Professional upgrade: Operational scripts need safe failure, evidence and rerunability; these habits transfer directly to scheduled pipelines.

10. Common beginner mistakes

1. except Exception: pass. | 2. Logging SUCCESS in finally even after failure. | 3. Dropping malformed rows without count/evidence. | 4. Writing output before checks then raising afterwards.

11. If you get stuck - recovery ladder

Recovery

Remove broad handlers. Let the original error surface. Add one specific handler only where you can improve the message or recover safely. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: inject missing file, missing column, duplicate master key and malformed numeric record; define stop/quarantine behavior for each.

13. Validate your result

Validation

Every failure message identifies file/rule/context; critical failure prevents final release file.

14. Teach it back in your own words

Explain without reading

Explain why 'the script did not crash' can still be a failed automation run.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry introduces a new bad record and requires quarantine or fail with evidence.

16. Evidence / mastery + exact next route

Mastery evidence

Failure matrix + code + captured messages + withheld-output proof + explain-back. After saving evidence, continue to PYA9.

PYA9 - Logging and validation checks

1. Why it matters

When an unattended job runs at 6 AM, you need evidence of what files it found, what it transformed, which checks passed and why output was released or withheld. Print statements alone are a poor operational history.

2. What you are learning

Configure meaningful logging and create machine-readable validation records containing check name, status, observed value and expectation/rule. Separate technical completion from business validation.

3. Prerequisite / what you already know

PYA8 defensive failure handling and basic dictionaries/JSON.

4. Picture it in your head

Mental model

Imagine this as a visible path: Run start -> Stage log -> Check records -> PASS/FAIL -> Output decision -> Run evidence. Keep the stages separate. If the result becomes wrong, walk backward one stage at a time until you find where the meaning, rule, grain, relationship, or control changed.

5. New words before we use them

Logger: Named logging object used to emit records at levels such as INFO, WARNING and ERROR. Validation record: Structured result of one business/data-quality check. Observability: Evidence that helps operators understand what a run did and where/why it failed.

6. Watch / visual source

VISUAL/SOURCE ROUTE: Corey Schafer Logging Basics - full targeted sections on levels, file output and formatting.

<https://docs.python.org/3.14/library/logging.html>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Configure logging with timestamp, level and message. Use a named logger.
- 2 Log meaningful checkpoints: run_start, files_discovered, rows_loaded, validation_complete, output_written/withheld.
- 3 Build checks as dictionaries with check, status, observed and rule/expected fields.
- 4 If a key-uniqueness or reconciliation check fails, log WARNING/ERROR and withhold final output even if the Python process technically completed other steps.
- 5 Write validation_results.json separately from the log so another process/person can inspect checks structurally.

8. What you should see / be able to say

Expected result

One log that reconstructs the run story plus a machine-readable validation file that explains PASS/FAIL.

9. Why it worked

Reasoning

Operational events and business trust checks are both recorded; neither substitutes for the other. Professional upgrade: Logging levels should tell the run story while structured validation supports automated gating/reporting. Do not log secrets or unnecessary row-level data.

10. Common beginner mistakes

1. Logging every row/line and hiding important events in noise. | 2. INFO says success although a critical validation failed. | 3. Validation file contains only PASS/FAIL with no observed/rule values. | 4. Secrets or raw sensitive payloads written into logs.

11. If you get stuck - recovery ladder

Recovery

List the 5-7 events a second analyst would need during incident recovery. Log only those plus warnings/errors; add structured checks for business trust. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: add logging and JSON validation to a customer enrichment run with duplicate-key, unmatched-key and total reconciliation checks.

13. Validate your result

Validation

Log has stage sequence; validation JSON has check/status/observed/rule; failed critical check blocks output.

14. Teach it back in your own words

Explain without reading

Explain the difference between 'script finished' and 'business validation passed'.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry adds a new validation rule without rewriting logging setup.

16. Evidence / mastery + exact next route

Mastery evidence

run.log + validation_results.json + withheld-output failure proof + explain-back. After saving evidence, continue to PYA10.

PYA10 - Automated recurring reporting

1. Why it matters

A weekly report is not automated merely because Python creates the file. The workflow must run from a clean start, be safe to rerun and avoid duplicating business rows or mutating raw inputs.

2. What you are learning

Orchestrate the module into one callable reporting workflow; externalize run period/paths; gate output on validation; make unchanged reruns reproduce/replace defined outputs safely.

3. Prerequisite / what you already know

PYA1-PYA9.

4. Picture it in your head

Mental model

Imagine this as a visible path: Config -> Discover -> Transform -> Validate -> Publish -> Rerun. Keep the stages separate. If the result becomes wrong, walk backward one stage at a time until you find where the meaning, rule, grain, relationship, or control changed.

5. New words before we use them

Idempotent-ish rerun: For this analyst context, rerunning the same approved inputs/config should not duplicate underlying business results or corrupt raw data. Orchestration: Coordinating smaller load/transform/validate/write functions in the required order. Release gate: A check that prevents publishing when critical validation fails.

6. Watch / visual source

VISUAL/SOURCE ROUTE: Book-led; logging and pathlib references support the workflow. Scheduling UI is optional and environment-specific.

<https://docs.python.org/3.14/library/logging.html>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Config supplies input_dir, customer_master, reporting_period and output_dir.
- 2 run_weekly discovers approved files, loads/cleans data and validates master-key assumptions before merge.
- 3 Use merge(..., validate='many_to_one') when that cardinality is the contract; surface unmatched keys.
- 4 Run structured checks. If any critical check is FAIL, raise/withhold final report.
- 5 Write report/log/validation/manifest to a defined run folder or deterministic filenames. Run twice on unchanged inputs and compare row counts/totals/hashes where appropriate.

8. What you should see / be able to say

Expected result

One callable weekly workflow that can be run manually from clean state and rerun without duplicate business data.

9. Why it worked

Reasoning

The orchestration calls tested components and final publishing depends on explicit trust checks. Professional upgrade: Use Windows Task Scheduler, cron or an orchestration platform only after the script has deterministic manual-run and rerun behavior. Scheduling is deployment, not correctness.

10. Common beginner mistakes

1. Scheduling before a clean manual run passes. | 2. Appending to last week's output blindly. | 3. Changing raw source files during cleaning. | 4. Duplicating transformation logic inside the scheduler entry point.

11. If you get stuck - recovery ladder

Recovery

Disable scheduling. Run manually from a clean folder. Prove first run, second unchanged run, and one validation-failure run before automating cadence. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: build a weekly support-performance workflow whose reporting period changes only in config and whose rerun replaces/reproduces the intended output safely.

13. Validate your result

Validation

Second unchanged run does not duplicate business rows; raw inputs unchanged; every stage traceable; critical validation gates publish.

14. Teach it back in your own words

Explain without reading

Explain why 'Task Scheduler launched Python' is not enough to call a reporting workflow reliable.

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry switches reporting period through config only.

16. Evidence / mastery + exact next route

Mastery evidence

Callable workflow + config + two-run comparison + failure-gate proof + explain-back. After saving evidence, continue to PYA11.

PYA11 - Reproducible analytical pipelines

1. Why it matters

A report that worked on your laptop is not reproducible if nobody knows which files, configuration, Python/pandas versions or dependency environment produced it.

2. What you are learning

Externalize configuration, use a project virtual environment, capture dependencies/versions and create a run manifest with inputs, row counts, checks and output hashes/paths.

3. Prerequisite / what you already know

PYA1-PYA10 complete workflow.

4. Picture it in your head

Mental model

Imagine this as a visible path: Code -> Config -> Environment -> Inputs -> Manifest -> Reproduce. Keep the stages separate. If the result becomes wrong, walk backward one stage at a time until you find where the meaning, rule, grain, relationship, or control changed.

5. New words before we use them

Virtual environment: Isolated Python environment for a project with its own installed packages. Dependency capture: Recording package requirements/versions needed to recreate the project environment. Run manifest: Machine-readable record of one run's configuration, inputs, versions, checks and outputs.

6. Watch / visual source

VISUAL/SOURCE ROUTE: Optional visual: Corey Schafer VENV on Windows; current Python 3.14 venv docs control commands.

<https://docs.python.org/3.14/tutorial/venv.html>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

Teacher demonstration - do this once with me, then close the example before your fresh task.

- 1 Move file paths/reporting period into config.json; transformation code reads config rather than embedding personal paths.
- 2 Create a project venv using the current Python 3.14 venv workflow on the learner machine; install only needed dependencies.
- 3 Capture Python, pandas and other key dependency versions in a reproducible form appropriate to the project/team.
- 4 At each run, record UTC timestamp, config identifier, exact input file names/sizes/hashes where practical, rows, validation results and output path/hash.
- 5 Move the project to a different parent folder or start a clean process and reproduce using configuration/root-path change only.

8. What you should see / be able to say

Expected result

Another analyst can identify and recreate the run inputs/environment and reproduce the same business output within defined controls.

9. Why it worked

Reasoning

The result is tied to explicit code/config/environment/input evidence rather than personal laptop state. Professional upgrade: Reproducibility is the bridge from analyst automation to engineering operations, incident recovery and later orchestration/CI practices.

10. Common beginner mistakes

1. requirements file never updated after package change. | 2. Absolute personal paths embedded in code. | 3. Manifest records output but not inputs/config/version. | 4. Calling a notebook reproducible because it was saved.

11. If you get stuck - recovery ladder

Recovery

Ask what a second machine needs: code, config, environment, inputs and run evidence. Add the first missing item before changing business logic. Save the genuine attempt before review.

12. Now you do a small version alone

Fresh independent task

Fresh case: package the weekly reporting workflow into project folders with config.json, requirements/dependency capture, outputs/logs and manifest. Reproduce from a changed root directory.

13. Validate your result

Validation

Config separate; versions recorded; exact inputs identifiable; second clean process/root reproduces row/total/output controls.

14. Teach it back in your own words

Explain without reading

Explain reproducibility as the ability to answer 'what exactly produced this result and can we recreate it?'

15. Protected review + changed fresh retry

Attempt protection

Attempt first. Changed retry moves the project root and modifies period in config only.

16. Evidence / mastery + exact next route

Mastery evidence

Project tree + config + venv/dependency evidence + run manifest + clean reproduction proof + explain-back. After saving evidence, complete the module mastery checklist and follow the Gate A / completion route.

C2-05 mastery checklist + quick reference

Use after the first pass; this does not replace running the automation

ELEVEN QUESTIONS

PYA1 Does it run from a clean process? | PYA2 Are function contracts visible/testable? | PYA3 Are dtypes/bad records/reconciliations explicit? | PYA4 Are expected files, lineage and schema drift controlled? | PYA5 Does exported Excel round-trip reconcile? | PYA6 What does one flattened JSON row mean? | PYA7 Can API failure/hang/HTTP error be distinguished from valid JSON? | PYA8 Does a critical failure stop release safely? | PYA9 Can logs + validation reconstruct trust? | PYA10 Can the same inputs rerun without duplicate business data? | PYA11 Can another machine/process identify and recreate the exact run?

BLOCK MASTERY IF

Hidden notebook state; hard-coded personal paths; silent coercion; uncontrolled file glob; lost lineage; output not re-read; wrong nested grain; API without timeout/status check; broad swallowed exception; validation failure ignored; rerun duplicates; no config/environment/input manifest.

FAST REPAIR LOOP

Clean process -> expected files -> schema/types -> transform -> key/grain -> independent totals -> failure injection -> withhold/release -> rerun -> manifest. Fix the first broken contract, not the last visible symptom.

DA -> DE BRIDGE

C2-05 intentionally introduces engineering habits such as deterministic ingestion, data-quality gates, observability, rerun safety and reproducibility. Those skills help C2B/C3, but taking C2-05 itself is not required for the direct fast route.

C2-05 source / video registry

Current official docs control behavior; selected videos are visual support

PYA1-2 - Python 3.14.7 modules

<https://docs.python.org/3.14/tutorial/modules.html>

Current modules/___main___ reference

PYA1,4,11 - Python 3.14.7 pathlib

<https://docs.python.org/3.14/library/pathlib.html>

Path-safe file operations

PYA1,4 - Real Python - Understanding Python File Paths

<https://realpython.com/videos/python-file-paths/>

00:00-02:10 short visual; optional

PYA1,4 - Corey Schafer - Pathlib modern file paths

<https://www.youtube.com/watch?v=yxa-DJuuTBI>

2024 optional reinforcement

PYA3-6,9-11 - pandas 3.0.5 User Guide

https://pandas.pydata.org/docs/user_guide/

Current pandas behavior

PYA3-4 - pandas.merge

<https://pandas.pydata.org/docs/reference/api/pandas.merge.html>

validate/cardinality/null-key behavior

PYA4-6 - pandas IO tools

https://pandas.pydata.org/docs/user_guide/io.html

CSV/JSON/Excel IO

PYA5-6 - Corey Schafer - pandas data sources

<https://www.youtube.com/watch?v=N6hyN6BW6ao>

CSV 00:56; Excel 06:15/10:42; JSON 12:18/15:41

PYA6 - Python csv module

<https://docs.python.org/3.14/library/csv.html>

Low-level CSV semantics

PYA6 - Python json module

<https://docs.python.org/3.14/library/json.html>

JSON parsing/serialization

PYA7 - Requests 2.34.2 Quickstart

<https://requests.readthedocs.io/en/latest/user/quickstart/>

timeout, raise_for_status, errors

PYA7 - Corey Schafer - Python Requests Tutorial

<https://www.youtube.com/watch?v=tb8gHvYICFs>

25 min visual; exact chapter metadata unavailable

PYA8 - Python 3.14 Errors and Exceptions

<https://docs.python.org/3.14/tutorial/errors.html>

Specific catches/raise/chaining

PYA9-10 - Python 3.14 logging

<https://docs.python.org/3.14/library/logging.html>

levels, handlers, operational messages

PYA9 - Corey Schafer - Logging Basics

<https://www.youtube.com/watch?v=-ARl4Cz-awo>

Visual levels/file logging/formatting

PYA11 - Python 3.14 venv

<https://docs.python.org/3.14/tutorial/venv.html>

Environment isolation/dependency workflow

PYA11 - Corey Schafer - VENV on Windows

<https://www.youtube.com/watch?v=APOPm01BVrk>

Optional Windows visual; current docs govern commands

VERIFY / FALLBACK

Verified 2026-09-10. If every video fails, the lesson remains complete. Official Python/pandas/Requests documentation wins over older video syntax.